

DAY-6

TASK : 5 Integrating AI Outputs with Backend APIs

The objective of this task was to start integrating AI-generated outputs with backend APIs so that pose detection results, body measurements, and recommendation data could be transferred between the AI model and the backend system in real time.

What I Worked On

During this task, I worked on connecting the AI processing module with backend APIs using Python and Flask. I integrated the pose detection and body measurement outputs generated using MediaPipe and OpenCV with a backend API created using Flask.

The AI model generated outputs such as:

- Shoulder width
- Hip width
- Estimated height
- Pose landmark information

These outputs were converted into JSON format and sent to the backend server using HTTP POST requests.

Integration Process I Followed

1. AI Processing

The webcam feed was processed using MediaPipe Pose Detection to extract body landmarks and estimate measurements.

2. Data Preparation

The generated measurements were stored inside a Python dictionary.

Example:

```
1 data = {  
2     "shoulder_width": shoulder_width_cm,  
3     "hip_width": hip_width_cm,  
4     "height": body_height_cm  
5 }
```

3. Backend API Creation

I created a backend API using Flask to receive AI-generated data.

4. API Communication

The AI outputs were sent to the backend using HTTP POST requests.

Example:

```
✓ response = requests.post(  
    "http://127.0.0.1:5000/bodydata",  
    json=data  
)
```

5. Backend Response

The backend server successfully received and displayed the JSON data in the terminal.

Technologies Used

- Python
- Flask
- MediaPipe
- OpenCV
- JSON
- HTTP Requests

Observations:

AI outputs were successfully transferred to the backend server.

JSON format made data communication simple and organized.

Flask APIs were easy to integrate with Python AI modules.

Real-time webcam processing could be connected directly with backend systems.

SCREENSHOTS:

1. FLASK SERVER RUNNING:

```
(venv) PS C:\Users\Akshith Kalvakota\Desktop\pose detection> python server.py
NameError: name 'shoulder_width_cm' is not defined
(venv) PS C:\Users\Akshith Kalvakota\Desktop\pose detection> python server.py
* Serving Flask app 'server'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 127-128-384
```

2. RECEIVED JSON DATA IS FLASK TERMINAL:

```
Received Data:
{'shoulder_width': 75.01, 'hip_width': 43.72, 'height': 226.05}
127.0.0.1 - - [18/May/2026 15:53:26] "POST /bodydata HTTP/1.1" 200 -

Received Data:
{'shoulder_width': 74.86, 'hip_width': 43.72, 'height': 231.9}
```

3. API SUCCESS RESPONSE:

```
(venv) PS C:\Users\Akshith Kalvakota\Desktop\pose detection> python bodymesaure.py
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
{'message': 'Data received successfully', 'received_data': {'height': 237.45, 'hip_width': 42.3, 'shoulder_width': 70.82}}
{'message': 'Data received successfully', 'received_data': {'height': 215.1, 'hip_width': 41.45, 'shoulder_width': 69.76}}
{'message': 'Data received successfully', 'received_data': {'height': 209.4, 'hip_width': 41.32, 'shoulder_width': 69.01}}
{'message': 'Data received successfully', 'received_data': {'height': 221.25, 'hip_width': 41.34, 'shoulder_width': 68.71}}
{'message': 'Data received successfully', 'received_data': {'height': 199.65, 'hip_width': 41.19, 'shoulder_width': 67.36}}
{'message': 'Data received successfully', 'received_data': {'height': 226.05, 'hip_width': 40.85, 'shoulder_width': 67.52}}
{'message': 'Data received successfully', 'received_data': {'height': 232.85, 'hip_width': 40.25, 'shoulder_width': 67.52}}
```

What I Learned

Through this task, I learned how AI models communicate with backend systems using APIs. I understood that backend integration is essential for transferring AI-generated outputs into real-world applications such as virtual try-on systems and recommendation platforms. I also learned how JSON data and HTTP requests are used to exchange information between AI modules and backend servers.